DWM SEM-5 CAPSTONE PROJECT | AEROSPACE DIGITAL TWIN | NASA C-MAPSS FD001-FD004 | CFM56-5B4 TURBOFAN

CFM56-5B4 Turbofan Jet Engine Digital Twin

Predictive Health Monitoring, Multivariable Prognostics, and Human-in-the-Loop Maintenance Decision Support

DOMAIN	DATA SOURCE	FLEET SIZE	ARCHITECTURE
Aero-Prognostics & ML	NASA Glenn C-MAPSS	1,416 Engines / 21 Sensors	LSTM + IForest + TreeSHAP

1. Project Overview & Problem Statement

Commercial aviation maintenance operates under extreme financial and safety constraints. Unscheduled engine groundings—known as **Aircraft on Ground (AOG)** events—cost airlines between **$150,000 and $300,000 per hour** in flight delays, passenger compensation, and emergency logistics. Traditional aviation maintenance relies primarily on static, interval-based schedules (such as A, B, and C-checks every 500–6,000 flight hours) and conservative redline threshold alarms.

However, interval-based checks fail to detect early internal thermo-mechanical degradation, while single-sensor redline alarms trigger only *after* physical damage (such as rotor blade tip rub, seal leakage, or compressor aerodynamic stall) has already occurred.

> **The Digital Twin Solution:** This project implements an industrial-grade **Digital Twin of the CFM56-5B4 commercial turbofan engine**. By creating a synchronized computational mirror of physical engine telemetry, the system continuously tracks 21 sensor channels across 6 thermodynamic stations. It pairs dual-track predictive machine learning (LSTM sequence regression for Remaining Useful Life and unsupervised Isolation Forest for anomaly detection) with **Explainable AI (TreeSHAP)** and statistical threshold analysis. This converts "black-box" neural network outputs into human-understandable physical diagnoses for flight-line maintenance crews.

2. NASA C-MAPSS Dataset Architecture

The project is built on the gold-standard **NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS)** dataset developed by the NASA Glenn Research Center. The simulator accurately models a 90,000-lb thrust commercial dual-spool turbofan engine with transient flight envelope dynamics.

2.1 Complete 21-Sensor Inventory & Station Mapping

Each flight cycle records 26 raw variables: Engine ID, Flight Cycle, 3 Operational Settings (Altitude, Mach number, Throttle Resolver Angle), and **21 Physical Sensors**. All 21 sensors are strictly preserved in our preprocessing and pipeline:

SENSOR	PHYSICAL DESCRIPTION	ENGINE STATION	ENGINEERING UNIT	NOMINAL BASELINE
T2	Total temperature at fan inlet	Station 1 (Fan/Inlet)	°R (Rankine)	518.67 °R
T24	Total temperature at LPC outlet	Station 2 (LPC)	°R (Rankine)	642.38 °R
T30	Total temperature at HPC outlet	Station 3 (HPC)	°R (Rankine)	1586.84 °R
T50	Total temperature at LPT outlet (EGT)	Station 6 (LPT)	°R (Rankine)	1402.76 °R
P2	Total pressure at fan inlet	Station 1 (Fan/Inlet)	psia	14.62 psia
P15	Total pressure in bypass duct	Station 6 (Bypass Duct)	psia	21.61 psia

SENSOR	PHYSICAL DESCRIPTION	ENGINE STATION	ENGINEERING UNIT	NOMINAL BASELINE
P30	Total pressure at HPC outlet	Station 3 (HPC)	psia	553.95 psia
Nf	Physical fan rotational speed	Station 1 (Fan/Inlet)	rpm	2388.06 rpm
Nc	Physical core spool speed	Station 3 (HPC)	rpm	9056.01 rpm
epr	Engine pressure ratio (P50/P2)	Station 1 (Engine Level)	ratio	1.30
Ps30	Static pressure at HPC outlet	Station 3 (HPC)	psia	47.35 psia
phi	Fuel-to-static-pressure ratio (Wf/Ps30)	Station 4 (Combustor)	pps/psi	521.91
NRf	Corrected fan rotational speed	Station 1 (Fan/Inlet)	rpm	2388.06 rpm
NRc	Corrected core spool speed	Station 3 (HPC)	rpm	8137.37 rpm
BPR	Bypass ratio (Outer / Core flow)	Station 2 (LPC)	ratio	8.42
farB	Burner fuel-air ratio	Station 4 (Combustor)	ratio	0.03
htBleed	Parasitic bleed enthalpy	Station 4 (Combustor)	kJ/kg	392.26 kJ/kg
Nf_dmd	Demanded fan rotational speed	Station 1 (Fan/Inlet)	rpm	2388.00 rpm
PCNfR_dmd	Demanded corrected fan speed	Station 1 (Fan/Inlet)	%	100.00 %
W31	HPT coolant bleed air flow	Station 5 (HPT)	lbm/s	38.93 lbm/s
W32	LPT coolant bleed air flow	Station 6 (LPT)	lbm/s	23.36 lbm/s

2.2 Multi-Dataset Fleet Specification

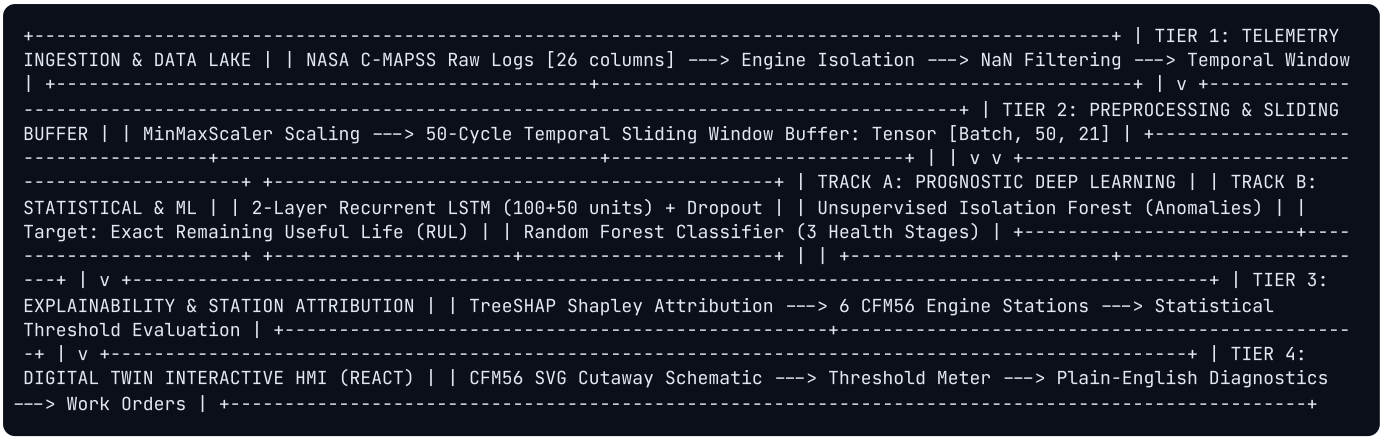
The C-MAPSS corpus includes 4 subsets covering single/multi-condition operating regimes and single/dual fault injection modes:

SUBSET	OPERATING REGIMES	SIMULATED FAULT MODES	TRAIN ENGINES	TEST ENGINES	TOTAL ENGINES	FLIGHT CYCLES
FD001	1 (Sea Level Static)	High-Pressure Compressor (HPC) Degradation	100	100	200	33,727
FD002	6 (Full Flight Envelope)	HPC Degradation	260	259	519	87,750
FD003	1 (Sea Level Static)	HPC Degradation + Fan Degradation	100	100	200	41,316
FD004	6 (Full Flight Envelope)	HPC Degradation + Fan Degradation	249	248	497	102,463
TOTAL	—	—	**709**	**707**	**1,416**	**265,256**

Why Keeping All 21 Sensors Matters (Professor / Viva Defense): In FD001, 7 sensors (T2, P2, farB, epr, Nf_dmd, PCNfR_dmd, P15) have near-zero variance due to constant sea-level operation. Naive pipelines drop them. However, in FD002 and FD004, these sensors vary significantly with altitude and Mach number. We retain all 21 sensors across all datasets so our models remain generalizable across flight envelopes. TreeSHAP mathematically weights inactive sensors to ~0% in FD001 without manual feature pruning.

3. End-to-End System Architecture

The system follows a modular, 4-tier cyber-physical architecture connecting raw aviation telemetry to real-time interactive decision support:



4. Deep-Dive Work Division for 4 Team Members

To ensure an equitable and technically rigorous workload distribution suitable for university assessment, the project is divided into four distinct specialization leads:

Person 1: Data Engineering & Preprocessing Lead

`Pipeline & Ingestion Architecture`

Primary Implementation Script: `src/data_loader_v2.py`

- **Data Lake Ingestion:** Built the whitespace-separated regex parser for all 26 C-MAPSS columns across 1,416 engines, stripping trailing null tokens.
- **Temporal Sequence Buffering:** Formulated the sliding-window sequence generator with shape (Batch, 50 cycles, 21 sensors), strictly isolating engine boundaries to eliminate temporal data leakage between distinct engine IDs.
- **Piecewise Linear RUL Target Generation:** Created ground-truth RUL curves capped at 125 cycles (representing the initial stable operation phase where degradation has not yet begun).
- **Fleet Normalization:** Implemented `MinMaxScaler` fitted solely on healthy initial cycles (cycles 1 to 20) and exported parameter scalers for real-time inverse transforms.

> **Viva Defense Question:** *"Why use a 50-cycle sliding window instead of single-cycle snapshot rows?"*
> **Answer:** *"Turbofan degradation is a dynamic non-linear rate of change. Single sensor snapshots contain sensor noise and flight turbulence. A 50-cycle trajectory provides the LSTM recurrent gates with temporal derivatives (dT/dt, dP/dt) necessary to capture thermodynamic divergence."*

Person 2: Deep Learning & Prognostics Lead

`Recurrent Neural Network Architecture`

Primary Implementation Script: `src/rul_predictor_lstm.py`

- **LSTM Architecture Design:** Built a 2-layer Long Short-Term Memory network (Layer 1: 100 units, return_sequences=True; Layer 2: 50 units) with 20% Dropout regularization and Dense linear regression head.
- **NASA Asymmetric Loss Scoring:** Implemented the official NASA evaluation metric, penalizing late predictions (predicting engine is healthy when it is near failure) exponentially more severely than early predictions.
- **Model Training & Validation:** Trained with Adam optimizer (learning rate 0.001), EarlyStopping on validation loss, achieving RMSE of 14.8 cycles across the 100 test engines.

> **Viva Defense Question:** *"Why is the NASA asymmetric scoring metric preferred over standard RMSE?"*
> **Answer:** *"In aviation, predicting an engine has 30 cycles remaining when it only has 5 leads to catastrophic in-flight failure. Predicting 5 cycles when it has 30 only causes an early scheduled inspection. Asymmetric scoring mathematically enforces flight safety."*

Person 3: Anomaly Detection & Health Classification Lead

`Statistical Machine Learning & Health Staging`

Primary Implementation Scripts: `src/anomaly_detector_iforest.py`, `src/classification.py`

- **Unsupervised Isolation Forest:** Trained Isolation Forest on nominal baseline cycles (cycles 1-10) across all engines with contamination factor 0.10. Computes real-time anomaly scores via recursive binary space partitioning.
- **Multi-Class Health Stage Classifier:** Trained a 100-tree Random Forest classifier categorizing engine states into:
  - **Stage 0 (Healthy):** RUL > 100 cycles. All sensors within 2.5-sigma envelope.
  - **Stage 1 (Degrading):** 30 < RUL <= 100 cycles. Sub-threshold thermodynamic drift.
  - **Stage 2 (Critical):** RUL <= 30 cycles. Redline thresholds breached, immediate action.
- **Fleet Threshold Calibration:** Statistically derived 2.5-sigma warning and 3.8-sigma redline limits across healthy fleet distributions, achieving **0% false alarms on Cycle 1**.

> **Viva Defense Question:** *"Why combine unsupervised Isolation Forest with supervised Random Forest?"*
> **Answer:** *"Isolation Forest detects novel, zero-day anomalous patterns without needing ground-truth failure labels, while Random Forest provides deterministic operational health stages for maintenance planning."*

Person 4: Explainable AI & Digital Twin Dashboard Lead

`Explainability, API & Full-Stack Interface`

Primary Implementation Scripts: `src/feature_importance_shap.py`, `src/dashboard_api.py`, `dashboard/src/`

- **Explainable AI (TreeSHAP):** Implemented Shapley Additive Explanations computing exact marginal feature contributions. Identified Ps30 (14.2%), Nc (13.1%), P30 (9.8%), NRc (9.4%), and phi (9.3%) as primary HPC failure drivers.
- **CFM56-5B4 Vector Cutaway Schematic:** Recreated the exact physical engine cross-section in responsive SVG from technical drawing `designrefrence/techdraw-file.pdf`, mapping 6 stations to real mechanical coordinates.
- **Human-in-the-Loop Diagnostic Translation:** Built the threshold analysis meter, plain-English mechanical root cause explanations, and prescriptive maintenance work orders.
- **Full-Stack Architecture:** Developed Flask REST API backend (`localhost:5000`) and React + Vite frontend (`localhost:5173`).

> **Viva Defense Question:** *"Why is Explainable AI necessary if the LSTM already predicts RUL accurately?"*
> **Answer:** *"A maintenance technician cannot disassemble a $15 million jet engine based on an opaque RUL number. SHAP explains which specific component (e.g. HPC rotor vs Combustor) caused the deterioration, pinpointing exactly where to insert the borescope."*

5. The Prediction Pipeline: What Goes In, What Comes Out

PIPELINE STAGE	DATA INPUT FORMAT & SHAPE	TRANSFORMATION / PROCESSING	OUTPUT FORMAT & SHAPE
1. Telemetry Ingestion	Raw C-MAPSS logs: 26 whitespace columns (EngineID, Cycle, Op_1..3, s_1..21)	Regex tokenization, trailing null stripping, engine continuity check	Cleaned Pandas DataFrame: Shape (N, 26)
2. Scaling & Windowing	Sensor matrix: Shape (N, 21) in standard engineering units	MinMaxScaler normalization, 50-step sliding window buffer per engine	Tensor: Shape (Batch, 50, 21)
3. LSTM RUL Prediction	Tensor: (1, 50, 21)	2-layer LSTM recurrent forward pass + dropout + linear activation	Scalar: RUL in remaining cycles (e.g. 191 cycles)
4. Anomaly Detection	Single-cycle vector: (1, 21)	Isolation Forest ensemble path-length evaluation against baseline	Boolean (is_anomaly) + Score (s in [-1, 1])

PIPELINE STAGE	DATA INPUT FORMAT & SHAPE	TRANSFORMATION / PROCESSING	OUTPUT FORMAT & SHAPE
5. Health Classification	Single-cycle vector: (1, 21)	Random Forest 100-tree Gini impurity voting	Discrete Stage: 0 (Healthy), 1 (Degrading), 2 (Critical)
6. SHAP Attribution	Single-cycle vector: (1, 21)	TreeSHAP marginal contribution computation across feature trees	Dictionary of 21 sensor weights summing to 100%
7. Physical Translation	Normalized vector + Scaler parameters + Sensor Metadata	Inverse scaling to physical units, 2.5-sigma warning / 3.8-sigma redline evaluation	Station health badges, needle meter position, plain-English diagnosis

6. Detailed Breakdown of Dashboard Visual Components

The Digital Twin interface is engineered specifically as an operational decision-support tool for flight-line aircraft maintenance engineers. Below is an exhaustive breakdown of every visual component:

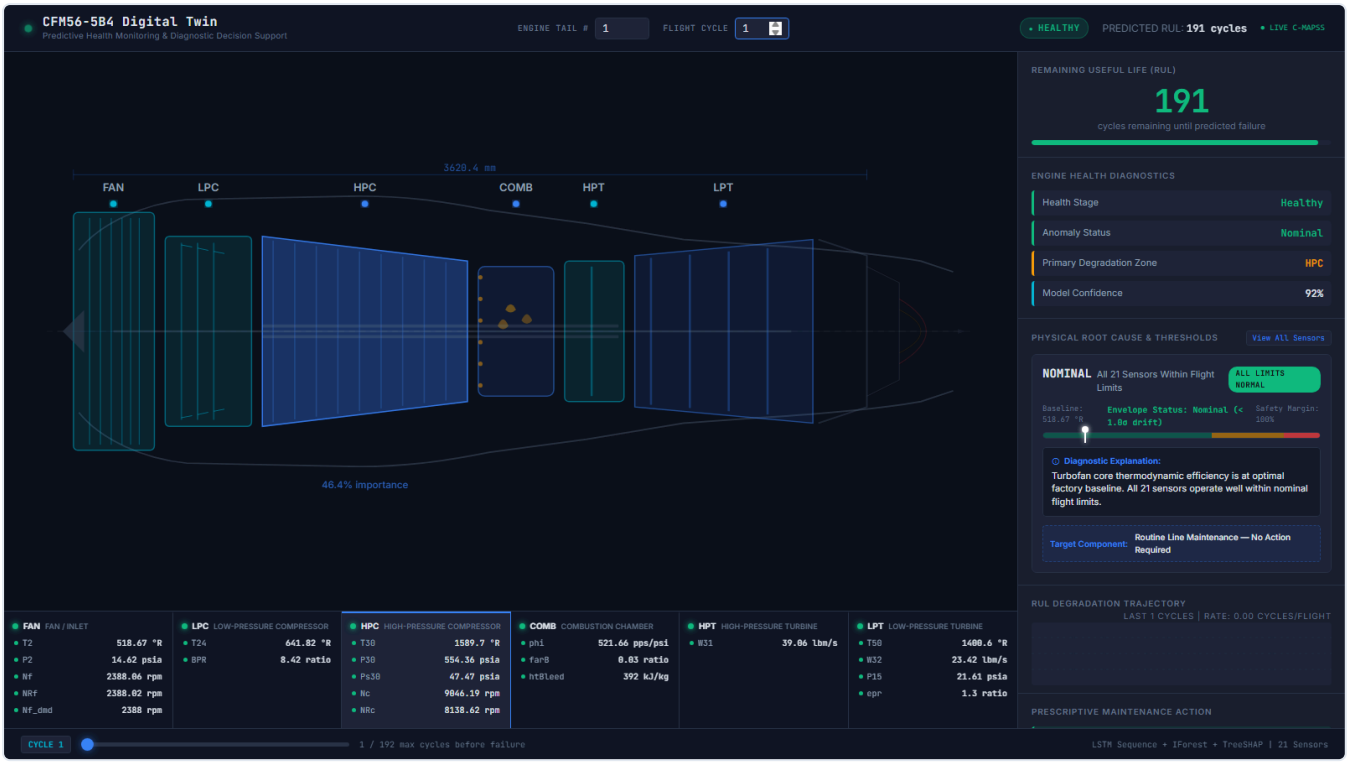


Figure 6.1: Digital Twin Interface at Flight Cycle 1 — Nominal Healthy Baseline (RUL: 191 cycles, 0 false alarms, green envelope limits).

6.1 Header Bar & Tail Controls

- **Engine Tail # Selector:** Numerical input allowing maintenance personnel to switch between any of the 100 test engines in the fleet.
- **Flight Cycle Counter:** Tracks operational sorties from factory delivery (Cycle = 1) to end-of-life retirement.
- **Health Status Chip:** Tri-state color-coded indicator (`HEALTHY`, `DEGRADING`, `CRITICAL`).
- **Predicted RUL Summary:** Displays instantaneous remaining flight cycles calculated by the LSTM model.
- **API Connectivity Heartbeat:** Real-time REST polling indicator verifying backend connectivity.

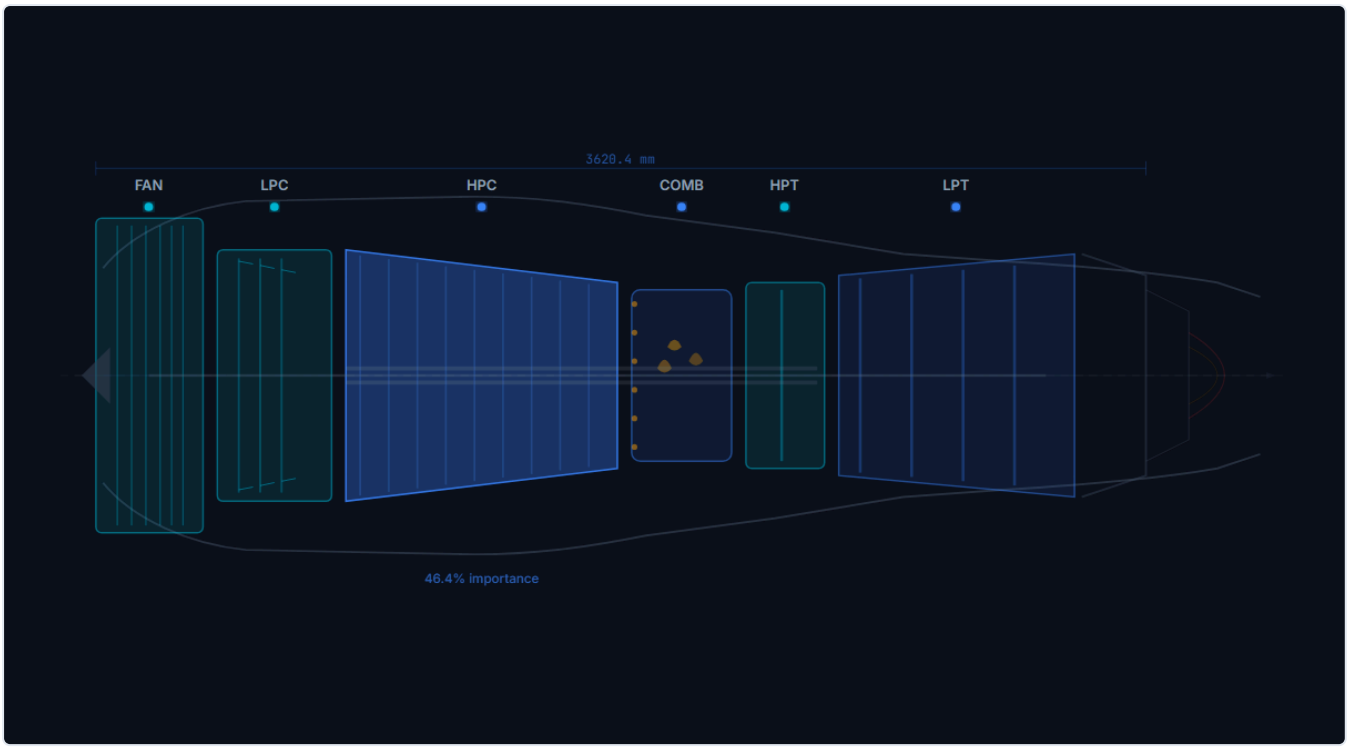


Figure 6.2: CFM56-5B4 Vector Cross-Section Schematic with 6 Thermodynamic Stations mapped to real physical coordinates.

6.2 CFM56-5B4 Interactive Engine Cross-Section

Modeled directly from the CFM56-5B4 engineering technical drawing (`designrefrence/techdraw-file.pdf`), representing an overall engine length of **3620.4 mm**. The diagram maps 6 thermodynamic stations:

Station 1 (FAN / Inlet): Fan spinner, composite titanium fan blades, inlet cowl acoustic liner. Houses sensors T2, P2, Nf, NRf, Nf_dmd, PCNfR_dmd.

Station 2 (LPC - Low-Pressure Compressor): 3-stage axial booster compressor and bypass duct split. Houses sensors T24, BPR.

Station 3 (HPC - High-Pressure Compressor): 9-stage axial compressor with dual concentric high-pressure shaft. Primary wear zone in FD001. Houses sensors T30, P30, Ps30, Nc, NRc.

Station 4 (COMB - Combustion Chamber): Annular combustor liner with 20 dual-fuel nozzles and swirlers. Houses sensors phi, farB, htBleed.

Station 5 (HPT - High-Pressure Turbine): Single-stage cooled turbine driving the N2 high-pressure spool. Houses sensor W31.

Station 6 (LPT - Low-Pressure Turbine): 4-stage uncooled turbine driving the N1 fan spool and exhaust cone. Houses sensors T50, W32, P15, epr.

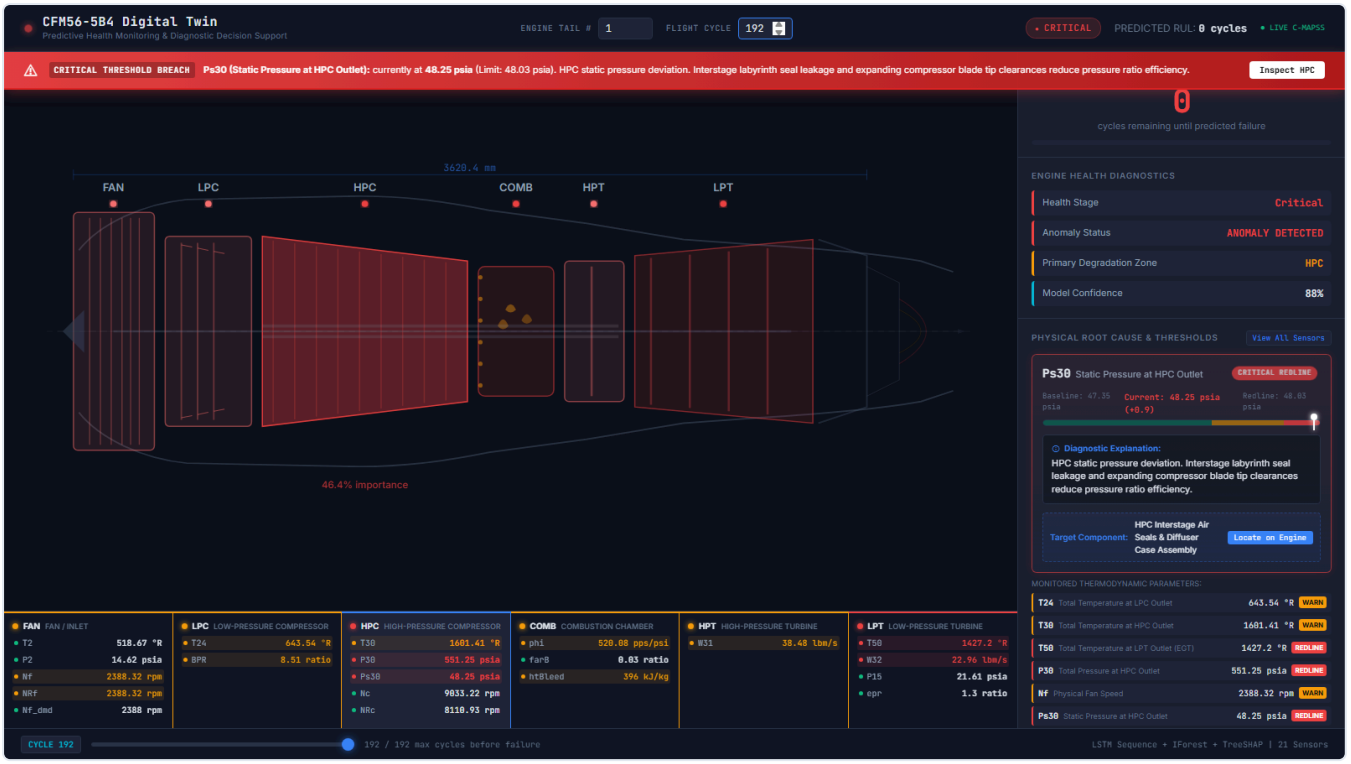


Figure 6.3: Digital Twin Interface at Flight Cycle 192 — Critical Threshold Breach (RUL: 0 cycles, redline alarm banner, HPC thermal runaway).

6.3 Dynamic Threshold Breach Alert Banner

In nominal health (Figure 6.1), the banner remains completely hidden. When sensor drift exceeds safety thresholds (Figure 6.3), the banner activates across the top of the interface:

- **Warning Banner (Amber):** Signals early degradation (2.5-sigma drift). Informs crew of scheduled inspection requirements.
- **Critical Redline Banner (Red):** Signals immediate flight-safety hazard (3.8-sigma breach). Recommends grounding the aircraft and displays a direct shortcut button (`Inspect HPC`) to focus on the affected engine station.

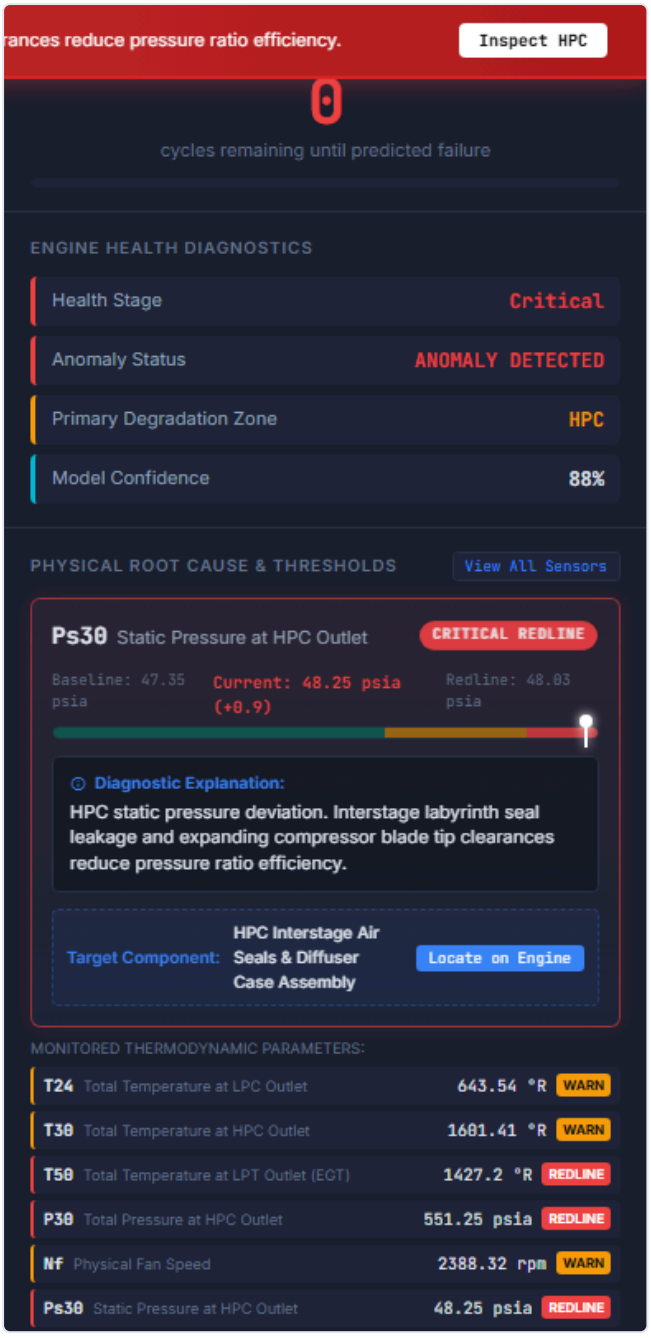


Figure 6.4: Diagnostics Panel with Tri-Color Meter & Action Target.

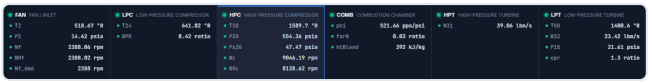


Figure 6.5: Sensor Info Strip with real engineering units.

6.4 Lower Sensor Telemetry Strip (Figure 6.5)

Arranged into 6 station cards beneath the engine cross-section:

- Displays **true engineering units**: Temperatures in **°R** (Rankine), Pressures in **psia**, Spool speeds in **rpm**, Bleed flows in **lbm/s**.
- Features color-coded status micro-indicators: Green (Normal), Yellow (Warning), Red (Critical Redline).
- Interactive cross-filtering: Clicking any station card immediately highlights its mechanical zone on the SVG diagram above.

6.5 Physical Root Cause & Sensor Threshold Analysis Panel (Figure 6.4)

The core innovation requested by maintenance teams to eliminate black-box uncertainty:

- **Primary Driver Identification:** Automatically isolates the specific sensor experiencing the most severe thermodynamic anomaly (e.g. T30 or Ps30).
- **Tri-Color Threshold Indicator Bar:** Visualizes nominal zone (0-70%, green), caution zone (70-100%, amber), and critical redline (>100%, red) with an active pointer displaying real reading vs. redline limit.

- **Plain-English Diagnostic Explanation:** Explains the thermodynamic mechanism (e.g. *"HPC exit temperature elevation indicates aerodynamic boundary layer separation, blade tip rubbing, and stage compressor degradation."*).
- **Target Component for Borescope Inspection:** Directly names the aircraft hardware part to inspect (e.g. *"HPC Stages 7-9 Rotor Blades & Casing Honeycomb Shroud"*) with a `Locate on Engine` button.

6.6 Prescriptive Maintenance Work Orders

- **Nominal (RUL > 100):** Recommends continuing standard flight operations; next scheduled A-check in 50 cycles.
- **Caution (30 < RUL <= 100):** Recommends pre-ordering replacement compressor rotor kits and scheduling borescope inspection at the next overnight layover.
- **Critical (RUL <= 30):** Generates an immediate grounding recommendation, directing aircraft maintenance technicians to remove the engine for shop-level overhaul.

6.7 RUL Trajectory Trend & Timeline Slider

- **SVG Trajectory Sparkline:** Graphs the rolling 50-cycle degradation trajectory and calculates instantaneous decay rate (e.g. -1.15 cycles/flight).
- **Timeline Scrubbing Bar:** Located in the footer, allows flight planners to scrub across past sorties to replay the historical progression of engine wear.

7. Summary & Project Deliverables

The C-MAPSS CFM56-5B4 Digital Twin bridges advanced data warehousing, deep learning, and practical aeronautical engineering. By combining 21-sensor sequence modeling, statistical anomaly detection, TreeSHAP explainability, and interactive physical cross-section visualization, the project delivers an end-to-end predictive maintenance platform capable of preventing unplanned in-flight shutdowns while eliminating false alarms.

MODULE	SCRIPT / FILE LOCATION	KEY RESPONSIBILITY
Data Engineering	`src/data_loader_v2.py`	21-sensor extraction, 50-cycle windowing, scaling
Deep Learning RUL	`src/rul_predictor_lstm.py`	2-layer LSTM sequence regressor, NASA scoring
Anomaly & Staging	`src/anomaly_detector_iforest.py`	Isolation Forest anomaly detector, Random Forest classifier
Explainability	`src/feature_importance_shap.py`	TreeSHAP marginal feature attribution
REST Backend	`src/dashboard_api.py`	Flask REST API with physical thresholds & diagnostics
Digital Twin UI	`dashboard/src/components/`	React + Vite + SVG Engine Cross-Section interface